O+C - Translate one input string into another.

Syntax:

```
O+C/str1/str2/
O-C
O-C/str/
```

Examples:

```
o+c/me/fbaggins/
o+c:\007:beep!:
o-c
o-c/m\Ce/
```

Options:

`/`
: It is general practice to use `/` as a delimiter in many commands. However, any non-alphanumeric character can also be used. The final delimiter may be omitted if `O-S/` is in effect and the `O+C` command is immediately followed by a new-line.

Description:

`O+C` tells FRED to translate the string `str1` into the string `str2` whenever `str1` is encountered in input. For example, if you said

```
o+c/me/fbaggins/
```

FRED would turn `me` into `fbaggins` whenever you typed it in, as a command or as input. Thus FRED would make the following conversions.

```
me/file --> fbaggins/file
mail to me --> mail to fbaggins
time --> tifbaggins
```

If special characters like `\B` appear in `str1` or `str2`, they are expanded at the time of the `O+C` command. Thus

```
o+c/abc/\b(xyz)/
```

tells FRED to convert `abc` into whatever is currently in buffer `(xyz)`. If the contents of `(xyz)` change later on, `abc` will still be converted into the old contents. On the other hand, if you say

```
o+c/abc/\C\B(xyz)/
```

FRED will change `abc` into `\B(xyz)`, which will then be replaced by whatever the contents of `(xyz)` are at the time.

Conversion takes place as soon as `str1` is recognized in your input. You will not see the change on your terminal screen, but the change will happen inside FRED.

Converted input is **not** scanned for additional conversions. Thus if you have

```
o+c/ABC/DEF/
o+c/DEF/GHI/
```

`ABC` will turn into `DEF`, but will not be converted again into `GHI`.

Usually, you will not convert strings of normal characters using `O+C`. Instead, you should think about binding special key sequences (like those from your terminal's function keys) to useful strings.

`O-C` stops a particular conversion. If you just say,

```
o-c
```

you cancel the effects of the most recent `O+C`. Cancelling a specific conversion is a little trickier. In essence, you say

```
o-c/str/
```

where `str` is the string that is being translated. The only problem is that `str` will be translated as you are entering the `O-C` command. Thus you have to put something in the middle of `str` so that it will not be translated. For example, you might say

```
o-c/a\Cbc/
```

to stop translating `abc`. The `\C` doesn't affect the meaning of the string `abc` but it ensures that FRED will not recognize the `abc` string and translate it.

Translations are kept in a "stack-like" list. Thus if you have

```
o+c/abc/def/
o+c/a\Cbc/ghi/
```

the second `O+C` command will override the first. If you use `O-C` to cancel the second translation, the first will become active again.

If `O-C` successfully deletes an `O+C`, it sets the condition register to TRUE. If it fails (i.e. the translation wasn't there), it sets the condition register to FALSE.

You can translate strings into null strings. For example,

```
o+c/\007//
```

tells FRED to ignore all `\007` characters that appear in input.

`FO` will print out all the translations you are currently using, in the order that you entered them in your FRED session.

Notes:

When FRED encounters a `V` command inside a translated string, FRED will perform the usual Voiding (undoing) action. In addition, FRED will check to see if the `V` undoes the `O+C` that asked for the translation in the first place. If it does, the rest of the translated string is discarded. For example, suppose you type in

```
o+c/\001/jm;hi; v jm;hello;/ \001
```

FRED will translate the `\001` and begin executing the commands. It does the first `JM`, finds the `V`, and voids the `O+C` that set the whole thing up. The second `JM` command will not be executed (as if it is no longer there).

Wild and wonderful side effects may occur if you try translating linefeeds or carriage returns into other things. Use at your own risk.